

AI/ML Engineer Interview Prep Guide

Simple explanations + how to answer in an interview

1. LLM (Large Language Models)

Q: What is a large language model?

Simple: A giant neural network trained on huge amounts of text (books, websites, code) to predict the next word in a sentence. Do that well enough, at a big enough scale, and it starts to “understand” language, answer questions, write code, etc.

Interview answer: “An LLM is a transformer-based neural network trained on massive text corpora to predict the next token given previous tokens. Scale (parameters + data + compute) is what gives it emergent abilities like reasoning and few-shot learning.”

Follow-up they may ask: Why transformers and not RNNs? → Because self-attention lets the model look at all words in parallel (fast training) and capture long-range relationships, whereas RNNs process word-by-word and forget older context.

Q: How are LLMs trained?

Simple: Three stages, like school:

1. **Pre-training** — read the entire internet, learn grammar, facts, patterns (self-supervised, just predicting next word).
2. **Fine-tuning / Instruction tuning** — teach it to follow instructions using labeled examples (prompt → ideal response).
3. **RLHF (alignment)** — polish it using human feedback so it’s helpful, honest, and safe.

Interview answer: “Pre-training on raw text builds general language ability, instruction tuning teaches it to follow tasks, and RLHF aligns its behavior with human preferences.”

Q: What is prompt engineering?

Simple: Carefully wording your input (prompt) to get the best output from the model —

without changing the model itself.

Interview answer: "It's the practice of designing inputs — instructions, examples, format constraints — to steer an LLM's output, instead of retraining it. Techniques include few-shot examples, chain-of-thought, role prompting, and output formatting (e.g., asking for JSON)."

Real-world tie-in (use this in interviews): In DocuMind, you likely wrote prompts telling the model to answer *only* from retrieved context and cite sources — that's prompt engineering in a RAG system.

Q: Few-shot vs zero-shot prompting?

Simple:

- **Zero-shot** = just ask, no examples. "Translate this to French: Hello"
- **Few-shot** = give 2-3 examples first, then ask. Helps the model understand the exact format/style you want.

Interview answer: "Zero-shot relies purely on the model's pretrained knowledge to follow an instruction with no examples. Few-shot provides a handful of input-output examples in the prompt to demonstrate the pattern, which improves accuracy on tasks with a specific format or domain."

Q: What is temperature in LLM generation?

Simple: A dial that controls randomness/creativity.

- Low temperature (0–0.3) → safe, predictable, focused answers (good for factual RAG answers, code).
- High temperature (0.7–1+) → creative, varied, sometimes weird answers (good for brainstorming, story writing).

Interview answer: "Temperature scales the probability distribution before sampling the next token. Lower temperature sharpens the distribution toward the most likely token (deterministic), higher temperature flattens it, increasing diversity/randomness."

Q: What is hallucination in LLMs?

Simple: The model confidently makes up facts, sources, or numbers that are false — because it's predicting plausible-sounding text, not looking things up.

Interview answer: "Hallucination happens because LLMs generate the statistically most likely next token, not verified facts. It's mitigated with RAG (grounding answers in retrieved documents), citations, and lower temperature — but never fully eliminated."

Tie-in: This is exactly why DocuMind's 100% RAGAS faithfulness score matters — it means answers were fully grounded in the retrieved documents, not hallucinated.

Q: What is instruction tuning?

Simple: Fine-tuning the base model on thousands of (instruction, ideal response) pairs so it learns to actually *follow* instructions instead of just completing text.

Interview answer: "It's supervised fine-tuning on instruction-response pairs, turning a raw next-token predictor into a model that behaves like an assistant that follows commands."

Q: What is RLHF (Reinforcement Learning from Human Feedback)?

Simple: After instruction tuning, humans rank multiple model answers (best to worst). A "reward model" learns from these rankings, and the LLM is further trained (via reinforcement learning) to produce answers the reward model scores highly.

Interview answer: "RLHF trains a reward model on human preference rankings, then uses RL (typically PPO) to fine-tune the LLM to maximize that reward — aligning outputs with human values like helpfulness and harmlessness."

Q: What are context windows in LLMs?

Simple: The maximum amount of text (measured in tokens) the model can "see" at once — your prompt + conversation history + any retrieved documents, all combined.

Interview answer: "The context window is the token limit the model can attend to in a single forward pass. Everything beyond it gets truncated or dropped, which is why RAG chunking and conversation memory management (like your ConversationBufferWindowMemory in DocuMind) matter — you have to fit the most relevant info in a limited space."

Q: What are embeddings and why are they used with LLMs?

Simple: Embeddings turn words/sentences into lists of numbers (vectors) that capture

meaning. Similar meanings → similar vectors, so a computer can “compare meaning” using math (distance between vectors) instead of exact word matching.

Interview answer: “Embeddings are dense vector representations of text produced by an encoder model. They let us do semantic similarity search — finding relevant text by meaning, not keyword match — which is the backbone of retrieval in RAG systems.”

Tie-in: You used HuggingFace’s `all-MiniLM-L6-v2` for this in DocuMind — good, specific detail to mention.

2. RAG (Retrieval Augmented Generation)

Q: What is RAG and why is it used?

Simple: Instead of relying only on what the LLM memorized during training, RAG first *retrieves* relevant chunks of text from your own documents/database, then feeds them to the LLM along with the question — so the answer is grounded in real, up-to-date, private data.

Interview answer: “RAG combines a retrieval system (vector search over your documents) with an LLM’s generation. It fixes the LLM’s two big weaknesses: outdated/limited knowledge and hallucination, by grounding answers in retrieved evidence.”

Q: What problem does RAG solve in LLM systems?

Simple: LLMs can’t know about your private documents, and they can’t be retrained every time your data changes — that’s slow and expensive. RAG lets you plug in fresh/private knowledge at query time without retraining.

Interview answer: “It solves the knowledge cutoff problem, the hallucination problem, and the cost problem — you don’t need to fine-tune a model every time your knowledge base changes; you just update the vector store.”

Q: What are embeddings in RAG pipelines?

Simple: Same concept as above — but here they’re used to convert both your documents and the user’s question into vectors, so you can find which document chunks are “closest” in meaning to the question.

Interview answer: “In RAG, an embedding model encodes each document chunk and the

incoming query into the same vector space, so similarity search (cosine similarity, typically) retrieves the most semantically relevant chunks."

Q: What is a vector database?

Simple: A special database built to store embeddings (vectors) and quickly find the "closest" ones to a given vector — instead of exact matches like a normal SQL database.

Interview answer: "A vector database stores high-dimensional embeddings and supports approximate nearest neighbor (ANN) search using algorithms like HNSW, enabling fast semantic search over millions of vectors."

Q: What are popular vector databases?

Simple list: ChromaDB, Pinecone, Weaviate, Qdrant, FAISS (library, not a full DB), Milvus, pgvector (Postgres extension).

Interview answer: "I've used ChromaDB in production (in DocuMind) for its simplicity and local persistence. Pinecone and Weaviate are popular managed/cloud options for larger scale; FAISS is a fast in-memory library often used for prototyping."

Q: What is chunking in RAG pipelines?

Simple: Splitting long documents into smaller pieces (chunks) before embedding them — because embeddings work better on focused pieces of text, and the LLM's context window can't fit a whole book anyway.

Interview answer: "Chunking breaks documents into overlapping segments (e.g., 500 tokens with 50-token overlap) before embedding. Chunk size is a key tuning parameter — too small loses context, too large dilutes relevance and wastes context window space."

Q: What is semantic search?

Simple: Searching by *meaning* instead of exact keywords. "How do I reset my password?" can match a document that says "steps to recover account access" — even though no words overlap — because their embeddings are close.

Interview answer: "Semantic search retrieves results based on vector similarity in embedding space, capturing meaning and context rather than lexical overlap, unlike

traditional keyword/BM25 search."

Q: How does retrieval work in RAG?

Simple pipeline:

1. User asks a question.
2. Question is embedded into a vector.
3. Vector DB finds the top-k closest document chunks.
4. Those chunks + the question are sent to the LLM as context.
5. LLM generates an answer grounded in that context.

Interview answer: Walk through the above steps clearly — interviewers love a clean pipeline explanation. Mention "top-k retrieval" and optionally "re-ranking" as a refinement step.

Q: How do you improve RAG accuracy?

Simple list of levers:

- Better chunking (size/overlap tuning)
- Better embedding model
- Hybrid search (keyword + semantic combined)
- Re-ranking retrieved chunks with a cross-encoder before sending to LLM
- Metadata filtering (e.g., only search a specific document/namespace)
- Prompt engineering (tell the LLM to only answer from context, cite sources)
- Evaluation with frameworks like RAGAS to measure faithfulness/relevancy and iterate

Interview answer: "I'd approach it systematically: measure first with RAGAS (faithfulness, answer relevancy, context precision/recall), then tune the weakest link — usually chunking strategy or retrieval quality — rather than guessing. In DocuMind, per-document namespace isolation also helped by preventing irrelevant cross-document retrieval, which is how I hit 100% faithfulness."

3. System Design & Production AI

Q: How would you design a scalable AI system?

Simple: Think in layers: API layer (FastAPI) → business logic → model/inference layer → data layer (vector DB, cache, storage). Add load balancing, caching, async processing, and horizontal scaling (more instances, not one giant server).

Interview answer: Structure your answer as: (1) clarify requirements/scale, (2) propose architecture diagram in words (API gateway → queue → workers → model serving → DB/cache), (3) call out bottlenecks (LLM inference latency, vector search speed), (4) mention caching, async/batching, and monitoring.

Q: How do you deploy machine learning models?

Simple: Package the model (Docker container), expose it via an API (FastAPI/Flask), host it (cloud VM, Kubernetes, or serverless), and add monitoring + versioning.

Interview answer: "I containerize the service with Docker for reproducibility, expose predictions via a FastAPI endpoint, and deploy behind a load balancer — as I did deploying DocuMind on Render. For larger scale I'd move to Kubernetes for auto-scaling."

Q: What is model monitoring in production?

Simple: Watching your model *after* deployment — not just accuracy, but latency, error rates, input data quality, and whether real-world data still looks like training data.

Interview answer: "Monitoring covers system metrics (latency, throughput, error rate), model quality metrics (accuracy proxies, user feedback/thumbs-down rate), and data metrics (drift detection) — because a model that worked at launch can silently degrade."

Q: What is model drift and data drift?

Simple:

- **Data drift** = the incoming data's statistical pattern changes over time (e.g., customer behavior shifts).
- **Model/concept drift** = the *relationship* between input and correct output changes (what used to be a "good" answer no longer is).

Interview answer: "Data drift is a change in input distribution; concept drift is a change in the underlying input-output relationship. Both cause performance decay and are why you need continuous monitoring and periodic retraining, not just a one-time deployment."

Q: How do you handle large scale inference?

Simple: Batch requests together, cache repeated queries, use smaller/quantized models where possible, use GPU inference servers, and scale horizontally (more workers) behind a queue.

Interview answer: Mention: request batching, model quantization, caching (especially for RAG — cache embeddings), async processing with queues (Celery/Redis), and autoscaling based on load.

Q: What is model versioning?

Simple: Keeping track of *which* version of a model (and its training data/config) is running in production — so you can roll back if a new version performs worse.

Interview answer: "Model versioning means tagging each trained model artifact with metadata (data version, hyperparameters, metrics) using tools like MLflow or DVC, so deployments are reproducible and reversible."

Q: What are A/B tests for ML models?

Simple: Show version A of the model to some users and version B to others, then compare real results (clicks, satisfaction, accuracy) to decide which is actually better in the real world.

Interview answer: "A/B testing splits live traffic between model versions to measure impact on real business/user metrics, which is more reliable than offline evaluation alone since it captures real user behavior."

Q: How do you optimize latency in AI systems?

Simple list: smaller/distilled models, quantization, caching, streaming responses (token-by-token), reducing prompt size, parallelizing retrieval + other steps, using faster hardware (GPU) or optimized inference engines (vLLM, TensorRT).

Interview answer: Pick 3-4 relevant to your project: "For DocuMind-style RAG systems, latency mostly comes from retrieval + LLM generation. I'd reduce it via smaller embedding models, caching frequent queries, streaming tokens to the UI so perceived latency drops, and using a fast inference provider (like Groq, which I used)."

Q: How would you build a real-time AI service?

Simple: Async API (FastAPI supports this natively), streaming responses, a message queue for spikes in traffic, and a fast model backend (like Groq's LPU inference, which is very low latency).

Interview answer: Describe: client → FastAPI (async) → queue if needed → model inference (streamed back via Server-Sent Events or WebSockets) → client sees tokens as they're generated.

Q: How do you ensure reliability of AI systems in production?

Simple: Retry logic, fallback models, timeouts, rate limiting, health checks, logging/alerting, graceful degradation (e.g., "answer not found" instead of crashing), and thorough testing before deploy.

Interview answer: "Reliability comes from defense in depth: timeouts and retries for external API calls, circuit breakers for downstream failures, structured logging for debugging, health checks for orchestration, and graceful fallback responses instead of hard failures."

4. ML & Deep Learning Fundamentals

Q: Supervised vs unsupervised vs reinforcement learning?

- **Supervised:** learn from labeled data (input → correct output given). E.g., spam detection.
- **Unsupervised:** find patterns in unlabeled data. E.g., clustering customers by behavior.
- **Reinforcement learning:** an agent learns by trial and error, getting rewards/penalties for actions. E.g., game-playing AI, RLHF for LLMs.

Interview answer: State the definitions crisply, then give one example each — interviewers want to see you can apply the concept, not just recite it.

Q: Overfitting and underfitting — how do you prevent them?

Simple:

- **Overfitting** = model memorizes training data, performs great on training but poorly on new data (too complex/learned noise).
- **Underfitting** = model is too simple, performs poorly even on training data.

Prevent overfitting: more data, regularization (L1/L2, dropout), simpler model, early stopping, cross-validation. **Prevent underfitting:** bigger/more complex model, train longer, better features.

Q: Bias vs variance tradeoff?

Simple:

- **High bias** = model is too simple, makes systematic errors (underfitting).
- **High variance** = model is too sensitive to training data, changes wildly with new data (overfitting).
- You want the sweet spot balancing both — total error = bias² + variance + irreducible noise.

Interview answer: Draw the classic U-shaped curve verbally: as model complexity increases, bias goes down but variance goes up; the best model minimizes total error, not just one of the two.

Q: Training, validation, and test datasets?

- **Training set** — model learns from this.
- **Validation set** — used to tune hyperparameters and check for overfitting *during* development.
- **Test set** — used only once, at the very end, to report true unbiased performance.

Interview tip: Emphasize you never touch the test set until the very end — leaking it during tuning is a classic mistake interviewers probe for.

Q: Gradient descent and its variants?

Simple: An optimization algorithm that adjusts model weights step-by-step in the direction that reduces error (loss), like walking downhill to find the lowest point.

Variants:

- **Batch GD** — uses the whole dataset per step (slow, stable).
 - **Stochastic GD (SGD)** — uses one example per step (fast, noisy).
 - **Mini-batch GD** — uses small batches (standard in practice — good balance).
 - **Adam, RMSprop** — adaptive variants that adjust the learning rate per parameter automatically, converge faster.
-

Q: Backpropagation — how does it work?

Simple: After a forward pass gives a prediction, backprop calculates how much each weight contributed to the error (using calculus — the chain rule) and pushes that error signal backward through the network, layer by layer, so gradient descent knows which direction to adjust each weight.

Interview answer: "Backpropagation applies the chain rule to compute the gradient of the loss with respect to every weight, propagating error backward from the output layer to the input layer, enabling gradient descent to update all weights efficiently in one pass."

5. Extra Questions Worth Prepping (AI/ML Engineer + Python Dev)

Since you're interviewing for **AI/ML Engineer, Python Developer, and NLP Engineer** roles, these commonly come up and weren't in your list:

LLM/GenAI extras

- What is LangChain / LangGraph and what problem does it solve? (orchestration framework for chaining LLM calls, tools, memory, retrieval)
- What is an AI agent, and how is it different from a simple LLM call? (agents can plan, use tools, and take multi-step actions, not just respond once)
- What is function calling / tool use in LLMs?
- What's the difference between fine-tuning and RAG — when would you pick one over the

other?

- What is quantization (e.g., 4-bit, 8-bit models) and why does it matter for deployment?
- What is LoRA / QLoRA (parameter-efficient fine-tuning)?

Python-specific (since you're also going for Python Developer roles)

- Difference between list, tuple, set, dict — and when to use each.
- What are Python generators and why use `yield`?
- What is the GIL (Global Interpreter Lock) and how does it affect multithreading?
- Difference between multiprocessing and multithreading in Python.
- What are decorators, and can you write a simple one?
- How does `async/await` work in Python (relevant since you use FastAPI)?
- What's the difference between `@staticmethod`, `@classmethod`, and instance methods?

System design extras

- How would you design DocuMind if it had to support 10,000 concurrent users? (great one to prep since it's your own project — talk about caching, load balancing, `async` FastAPI, sharding the vector DB)
- What's the difference between horizontal and vertical scaling?
- How do you handle secrets/API keys securely in production (env vars, secret managers)?

Behavioral (STAR) — since you're building STAR answers already

- "Tell me about a time you had to debug a tricky production issue" (use ETHAN's latency optimization: 50% API latency improvement)
- "Tell me about a project you're proud of" (DocuMind — mention the 100% RAGAS faithfulness metric and *why* you built per-document namespace isolation, since explaining a design decision shows depth)
- "How do you keep up with fast-moving AI trends?"

Quick Tips for the Interview Itself

1. **Always tie answers to your own projects** (DocuMind, ETHAN, IDE) when possible —

numbers like "100% RAGAS faithfulness" and "50% latency improvement" stand out far more than textbook definitions alone.

2. When asked to define something, give the **simple version first, then the precise/technical version** — shows you understand it, not just memorized it.
3. For system design questions, **always clarify scale/requirements first** before jumping into an architecture — interviewers notice this.
4. If you don't know something, say what you *do* know that's adjacent, rather than freezing — partial credit matters.